

STRUCTS

LINGUAGEM DE PROGRAMAÇÃO

TALITA BERBEL

talita.berbel@fatec.sp.gov.br

Fatec
Sorocaba
José Crespo Gonzales

CP
Centro
Paula Souza

AGENDA

- Introdução
- Definição
- Declaração
- Acesso
- Atribuição
- Structs aninhadas
- Vetor de Structs
- Dúvidas



INTRODUÇÃO

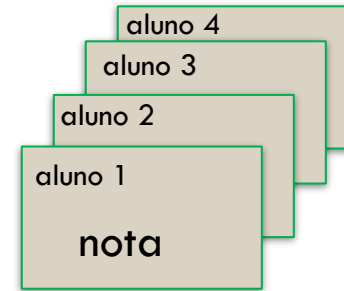
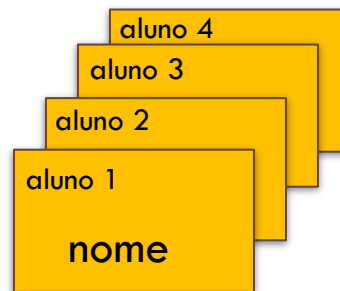
- Uma **estrutura** ou **struct** é uma **coleção** de **uma ou mais variáveis**, possivelmente de **tipos diferentes**, colocadas juntas sob um **único nome**.
- Existem linguagens que chamam as estruturas de registros.
- Ideal para representar entidades do mundo real, como um aluno, produto ou cliente.
- Sendo assim, uma estrutura é um **tipo de dado** cujo formato é definido pelo(a) programador(a).



INTRODUÇÃO

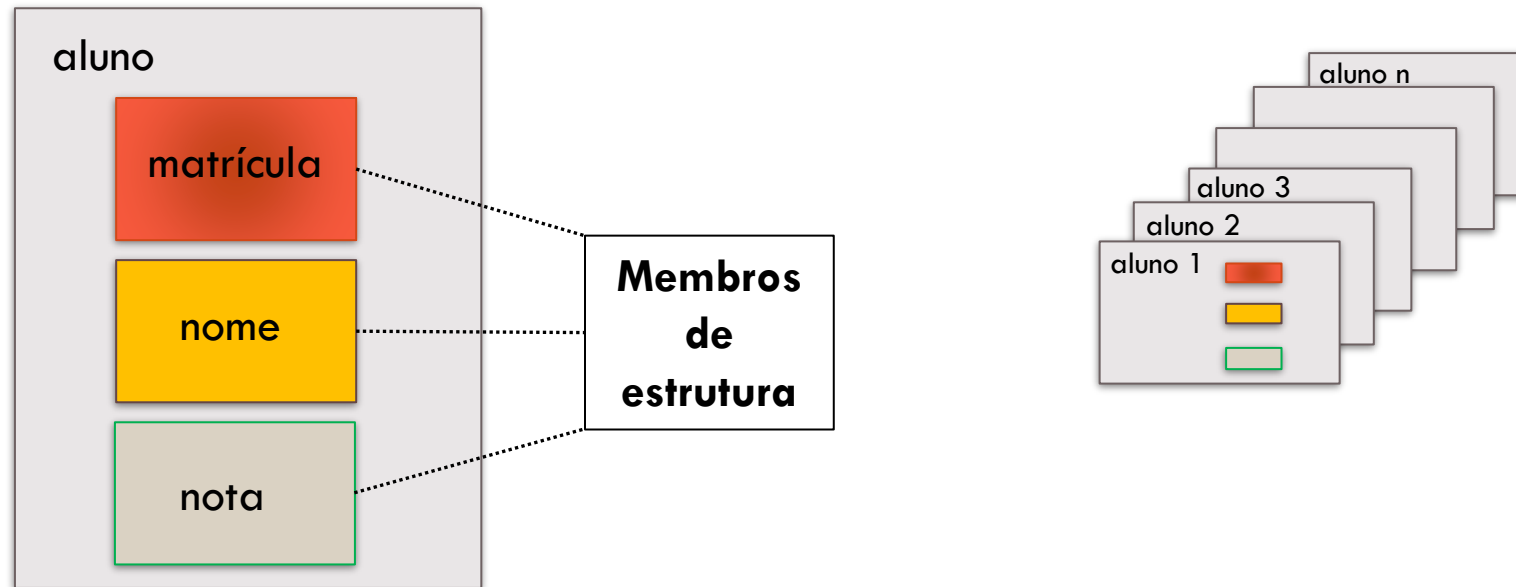


- Exemplo: Receber e armazenar matrícula, nome e uma nota de alunos de uma turma.
- A matrícula pode ser do tipo inteiro, já nome seria tipo string e nota, tipo float.



INTRODUÇÃO

- Exemplo da estrutura **aluno** com matrícula, nome e nota:



Uma estrutura consiste de um certo número de dados, chamados membros da estrutura, que não necessitam ser de mesmo tipo, agrupados juntos.

INTRODUÇÃO

- Para utilizar estrutura em um programa é necessário:
 - **Definir** a estrutura – ocorre depois do `#define` e antes dos protótipos das funções.
 - **Declarar** uma variável, vetor ou ponteiro do tipo da estrutura – pode ser na `main()`, dentro de outras funções ou nos parâmetros das funções.
 - **Acessar os membros** da estrutura – em qualquer função do programa, ou seja, na `main()` ou demais funções.

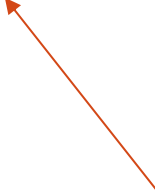


DEFINIÇÃO

- A **definição da estrutura** ocorre antes da função principal e não reserva espaço de memória. Ela determina o nome e os membros da estrutura.
- A palavra struct define o novo tipo, o seu nome é chamado de rótulo e os membros da struct ficam entre parênteses.

- **Sintaxe:**

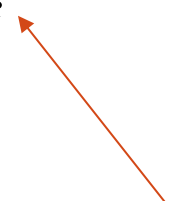
```
struct nomeStruct {  
    tipo1 campo1;  
    tipo2 campo2;  
};
```



Finalizar com ponto e vírgula

- **Exemplo:**

```
struct aluno {  
    int matricula;  
    char nome[50];  
    float nota;  
};
```



Podem ser variáveis
simples ou vetores

DECLARAÇÃO

- A **declaração da variável do tipo da estrutura** ocorre dentro da função principal ou outra função. Pode ser reservado espaço de memória para uma estrutura ou um conjunto (vetor) do tipo da estrutura.

- **Sintaxe:**

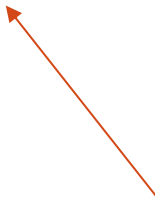
```
struct nome_struct variável;
```

```
struct nome_struct vet[10];
```

- **Exemplo:**

```
struct aluno a1;
```

```
struct aluno turma[40];
```



Vetor de estruturas

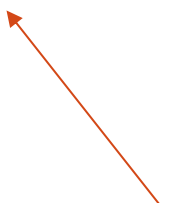
DECLARAÇÃO

- Para facilitar o tratamento das structs durante o programa, pode-se utilizar o comando **typedef**, assim não é necessário escrever struct antes de mencionar o nome da struct em qualquer parte do programa.

- **Sintaxe:**

```
typedef struct nomeStruct {  
    tipo1 campo1;  
    tipo2 campo2;  
}nomeStruct;
```

Nome do apelido da estrutura



- **Exemplo:**

```
typedef struct aluno {  
    int matricula;  
    char nome[50];  
    float nota;  
}aluno;  
  
//declarando variáveis  
aluno a1, turma[40];
```

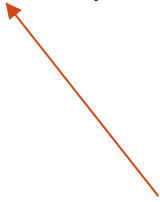
DECLARAÇÃO

- Assim como as demais variáveis seria possível declarar e inicializar os membros da estrutura.

- Sintaxe:**

```
typedef struct nomeStruct {  
    tipo1 campo1;  
    tipo2 campo2;  
}nomeStruct;
```

```
//dentro de uma função  
nomeStruct var = {v1,v2};
```



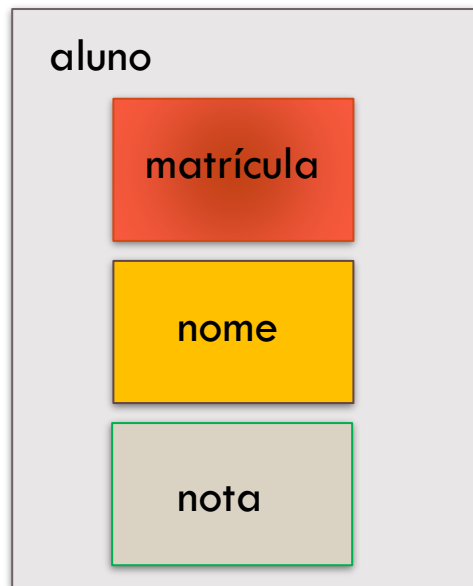
Conteúdo campo1 e campo2

- Exemplo:**

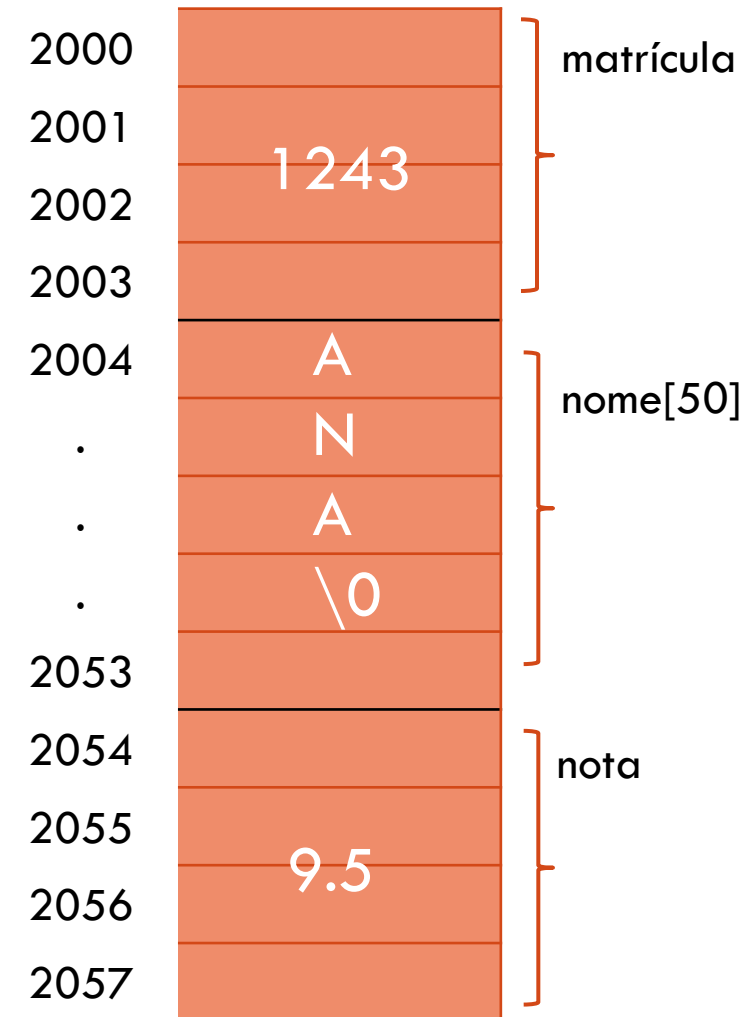
```
typedef struct aluno {  
    int matricula;  
    char nome[50];  
    float nota;  
}aluno;  
  
//declarando e inicializando  
aluno a1 = {1234,"Ana",9.5};  
aluno a2 = {1235,"Pedro",8.75};
```

DECLARAÇÃO

- Exemplo da variável **aluno** na memória.



```
typedef struct aluno {  
    int matricula; //4 bytes  
    char nome[50]; //50 bytes  
    float nota; //4 bytes  
}aluno;  
//declarando e inicializando  
aluno a1 = {1234,"Ana",9.5};
```

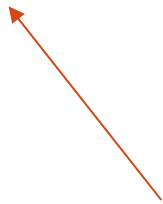


ACESSO

- Para **acessar os membros da struct** durante o programa, utiliza-se o operador de associação ou operador “ponto” logo após o nome da variável do tipo da estrutura.

- Sintaxe:

`variável.campo1;`



Operador de associação

- Exemplo:

```
aluno a1;  
//inicializando os membros  
a1.matricula=1234;  
strcpy(a1.nome, "Ana");  
a1.nota=9.5;
```

ACESSO

- **Exemplo:**

```
#include <stdio.h>
#include <string.h>
typedef struct aluno {
    int matricula;
    char nome[50];
    float nota;
}aluno;
```

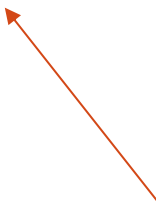
```
int main()
{
    aluno a1 = {1234, "Ana", 9.5};
    printf("\nMat.: %i", a1.matricula);
    printf("\nNome: %s", a1.nome);
    printf("\nNota: %.2f", a1.nota);
}
```

ATRIBUIÇÃO

- Diferente da atribuição entre strings ou vetores, é possível realizar a **atribuição** de variáveis do tipo da estrutura, apenas com o operador de atribuição (=). Desse modo, todos os valores na estrutura estão realmente sendo atribuídos para os correspondentes elementos da outra estrutura

- Sintaxe:

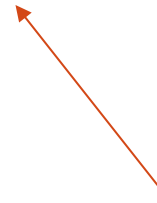
```
variável_a=variável_b;
```



Operador de atribuição

- Exemplo:

```
aluno a1 = {1234, "Ana", 9.5};  
aluno a2;  
a2 = a1;
```



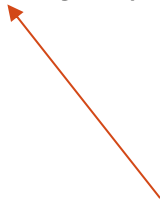
Os dados para os dois alunos serão exatamente os mesmos.

ATRIBUIÇÃO

- A **atribuição dos membros** de uma variável do tipo da estrutura, é possível mas é necessário considerar o tipo do membro da estrutura.

- Sintaxe:

```
var.campo1=var.campo2;
```



Para valores numéricos e
char simples

- Exemplo:

```
aluno a1 = {1234, "Ana", 9.5};  
aluno a2;  
a2.matricula=a1.matricula;  
a2.nota=a1.nota;  
//copiando primeira letra  
a2.nome[0]=a1.nome[0];
```

```
//copiando nome com strcpy()  
strcpy(a2.nome, a1.nome);
```

STRUCTS ANINHADAS

- É possível que dentre os tipos dos membros da struct possam existir o tipo de uma outra struct.

- **Sintaxe:**

```
typedef struct nomeStruct {  
    tipo1 campo1;  
    tipo2 campo2;  
}nomeStruct;
```

```
typedef struct nomeStruct2 {  
    tipo3 campo3;  
    nomeStruct campo4;  
}nomeStruct2;
```

- **Exemplo:**

```
typedef struct graduacao{  
    int id;  
    char periodo;  
}graduacao;
```

```
typedef struct aluno {  
    int matricula;  
    char nome[50];  
    float nota;  
    graduacao curso;  
}aluno;
```

STRUCTS ANINHADAS

- Exemplo:

```
typedef struct graduacao{
    int id;
    char periodo; //D ou N
}graduacao;

typedef struct aluno {
    int matricula;
    char nome[50];
    float nota;
    graduacao curso;
}aluno;
```

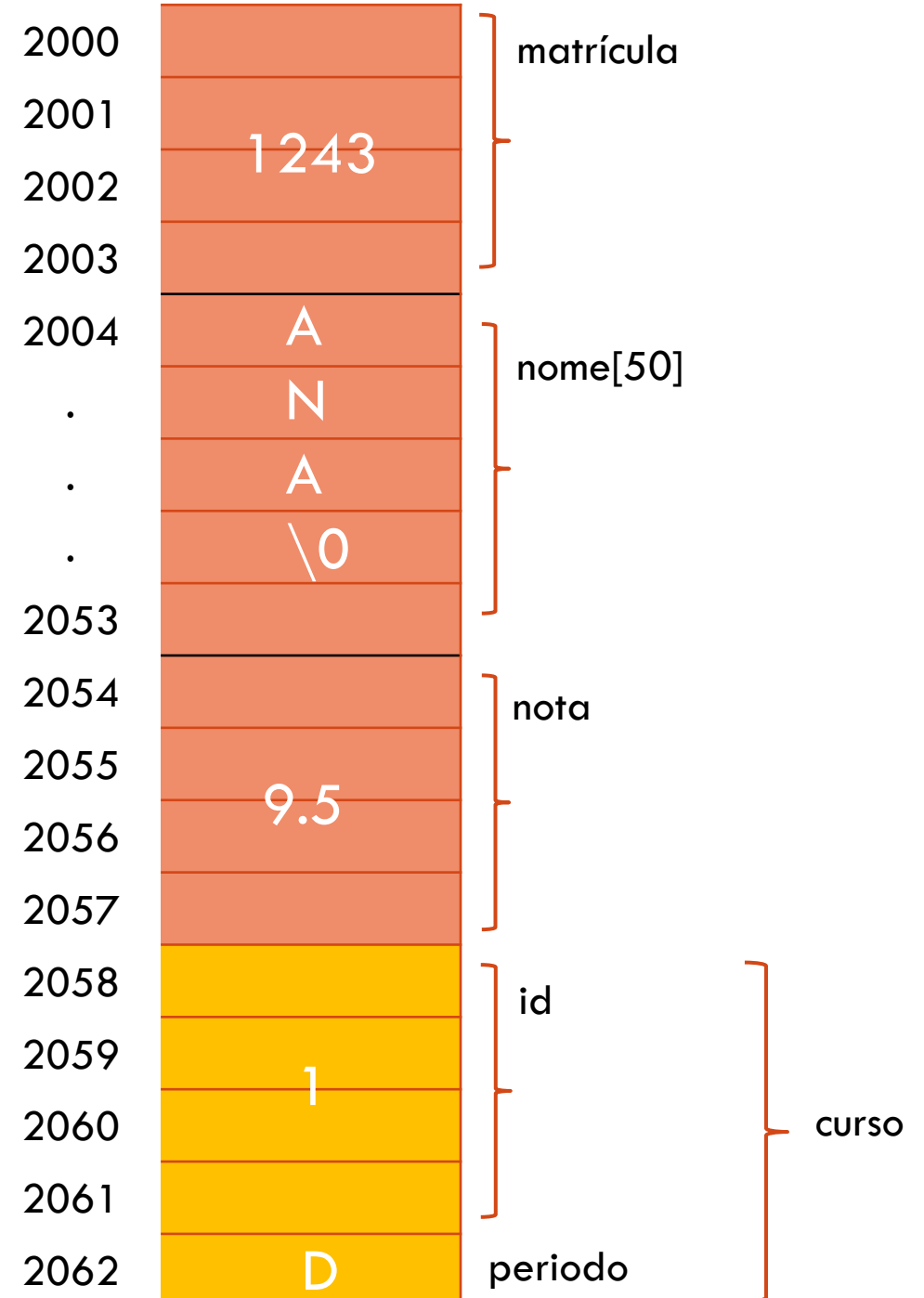
```
int main()
{
    aluno a1 = {1234, "Ana", 9.5, {1, 'D'}};
    printf("\nMat.: %i", a1.matricula);
    printf("\nNome: %s", a1.nome);
    printf("\nNota: %.2f", a1.nota);
    printf("\nCurso: %i", a1.curso.id);
    printf("\nPer.: %c", a1.curso.periodo);
}
```

STRUCTS ANINHADAS

- Exemplo:

```
typedef struct graduacao{  
    int id;  
    char periodo;  
}graduacao;
```

```
typedef struct aluno {  
    int matricula;  
    char nome[50];  
    float nota;  
    graduacao curso;  
}aluno;
```



STRUCTS ANINHADAS

- **Exemplo:**

```
aluno a1,a2;  
a1.matricula=1234;  
strcpy(a1.nome,"Ana");  
a1.nota=9.5;  
a1.curso.id=1;  
a1.curso.periodo='D';
```

```
a2.matricula=1235;  
strcpy(a2.nome,"Pedro");  
a2.nota=8.75;  
a2.curso.id=a1.curso.id;  
a2.curso.periodo=a1.curso.periodo;
```

```
printf("\n%i %s %f %i %c",a1.matricula,a1.nome,a1.nota,a1.curso.id,a1.curso.periodo);  
printf("\n%i %s %f %i %c",a2.matricula,a2.nome,a2.nota,a2.curso.id,a2.curso.periodo);
```

Na tela:

```
1234 Ana 9.500000 1 D  
1235 Pedro 8.750000 1 D
```

STRUCTS ANINHADAS

- **Exemplo:**

```
aluno a1,a2;  
a1.matricula=1234;  
strcpy(a1.nome,"Ana");  
a1.nota=9.5;  
a1.curso.id=1;  
a1.curso.periodo='D';
```

```
a2.matricula=1235;  
strcpy(a2.nome,"Pedro");  
a2.nota=8.75;  
a2.curso=a1.curso;
```

```
printf("\n%i %s %f %i %c",a1.matricula,a1.nome,a1.nota,a1.curso.id,a1.curso.periodo);  
printf("\n%i %s %f %i %c",a2.matricula,a2.nome,a2.nota,a2.curso.id,a2.curso.periodo);
```

Na tela:

```
1234 Ana 9.500000 1 D  
1235 Pedro 8.750000 1 D
```

STRUCTS ANINHADAS

- **Exemplo:**

```
aluno a1,a2;
a1.matricula=1234;
strcpy(a1.nome,"Ana");
a1.nota=9.5;
a1.curso.id=1;
a1.curso.periodo='D';

a2.matricula=1235;
strcpy(a2.nome,"Pedro");
a2.nota=8.75;
a2.curso=a1.curso;

imprime(a1);
imprime(a2);
```

```
void imprime(aluno aux)
{
    printf("%i ",aux.matricula);
    printf("%s ",aux.nome);
    printf("%f ",aux.nota);
    printf("%i ",aux.curso.id);
    printf("%c\n",aux.curso.periodo);
}
```

DIFERENÇAS ENTRE ARRAYS E STRUCTS

Aspecto	Array (Vetor ou Matriz)	Struct (Estrutura)
Definição	Coleção de elementos do mesmo tipo , armazenados de forma sequencial na memória.	Coleção de elementos de tipos diferentes , agrupados em uma única unidade lógica.
Tipos de Arranjos	Homogêneos : todos os elementos devem ser do mesmo tipo.	Heterogêneo : cada campo pode ter um tipo diferente.
Acesso aos elementos	Acesso por índice , começando de 0. Ex.: numeros[0], numeros[1].	Acesso por nome , usando . (ponto). Ex.: aluno1.matricula, aluno1.nome.
Itens de Dados	Elementos do vetor.	Membros da struct.
Organização dos dados	Estrutura linear , ideal para listas e coleções de valores iguais. Ex.: lista de notas, strings.	Estrutura lógica , ideal para representar entidades do mundo real, como "aluno" e "produto". Ex.: cadastro de alunos.
Memória	Espaço contínuo, cada elemento ocupa exatamente o tamanho do tipo definido.	Espaço contínuo, mas campos podem ter tamanhos diferentes . Podem existir espaços extras (<i>padding</i>).
Complexidade	Mais simples de declarar e manipular.	Mais complexo , mas mais organizado para dados reais.

VETOR DE STRUCT

- O **vetor da estrutura** pode ser declarado em qualquer função e também pode ser enviado como referência para uma função.

- **Sintaxe:**

```
typedef struct nomeStruct {  
    tipo1 campo1;  
    tipo2 campo2;  
}nomeStruct;
```

```
//dentro de uma função  
nomeStruct vetor[20];
```



Quantidade de elementos

- **Exemplo:**

```
typedef struct aluno {  
    int matricula;  
    char nome[50];  
    float nota;  
}aluno;
```

```
//dentro de uma função  
aluno turma[30];  
aluno lista1[2];
```

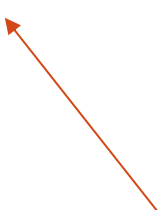
VETOR DE STRUCT

- O **elementos do vetor da estrutura** pode ser acessados considerando a sua posição dentro do vetor.

- **Sintaxe:**

```
typedef struct nomeStruct {  
    tipo1 campo1;  
    tipo2 campo2;  
}nomeStruct;
```

```
//dentro de uma função  
nomeStruct vetor[20];  
vetor[0].campo1=v1;
```



Conteúdo do membro

- **Exemplo:**

```
typedef struct aluno {  
    int matricula;  
    char nome[50];  
    float nota;  
}aluno;
```

```
//dentro de uma função  
aluno turma[30];  
turma[0].matricula=123;
```

VETOR DE STRUCT

- **Exemplo: Apenas um elemento**

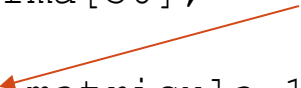
```
typedef struct aluno {
    int matricula;
    char nome[50];
    float nota;
}aluno;

int main() {
    aluno turma[30];

    turma[0].matricula=1234;
    strcpy(turma[0].nome, "Ana");
    turma[0].nota=9.5;
    turma[0].curso.id=1;
    turma[0].curso.periodo='D';

    imprime(turma[0]);
}
```

Elemento 0



```
void imprime(aluno aux)
{
    printf("%i ",aux.matricula);
    printf("%s ",aux.nome);
    printf("%f ",aux.nota);
    printf("%i ",aux.curso.id);
    printf("%c\n",aux.curso.periodo);
}
```

VETOR DE STRUCT

- **Exemplo:**

```
void imprime(aluno aux);
aluno cadastro(int i);
int main()
{
    aluno turma[3];
    int i;

    for(i=0;i<3;i++){
        turma[i]=cadastro(i+1);
    }

    for(i=0;i<3;i++){
        imprime(turma[i]);
    }
    return 0;
}
```

 **Acessando 3 elementos**

```
aluno cadastro(int i)
{
    aluno aux;
    printf("\nMatr %i :",i);
    scanf("%i",&aux.matricula);
    fflush(stdin);
    printf("Nome %i: ",i);
    gets(aux.nome);
    printf("Nota %i: ",i);
    scanf("%f",&aux.nota);
    return aux;
}

void imprime(aluno aux)
{
    printf("%i ",aux.matricula);
    printf("%s ",aux.nome);
    printf("%f\n",aux.nota);
}
```

VETOR DE STRUCT

- Exemplo: Utilizando Ponteiro

```
typedef struct aluno {  
    int matricula;  
    char nome[50];  
    float nota;  
}aluno;  
void imprime(aluno *p);  
void cadastro(aluno *p);
```

```
int main()  
{  
    aluno turma[3];  
    cadastro(turma);  
    imprime(turma);  
    return 0;  
}
```

Guardando end.
de turma



```
void cadastro(aluno *p){  
    int i;  
    for(i=0;i<3;i++){  
        printf("\nMatr %i :",i+1);  
        scanf("%i",&(p+i)->matricula);  
        fflush(stdin);  
        printf("Nome %i: ",i);  
        gets((p+i)->nome);  
        printf("Nota %i: ",i);  
        scanf("%f",&(p+i)->nota);  
    }  
}  
void imprime(aluno *p){  
    int i;  
    for(i=0;i<3;i++){  
        printf("%i ",(p+i)->matricula);  
        printf("%s ",(p+i)->nome);  
        printf("%f\n", (p+i)->nota);  
    }  
}
```

DÚVIDAS

- Questionamentos?

